Explore, Conjecture, Discover: Experimental Mathematics with Mathematica and Python

A Guide to Mathematical Research for High School Students

James H. Choi, Ph.D.

2026-04-09

Table of contents

Preface

This is a Quarto book.

To learn more about Quarto books visit https://quarto.org/docs/books.

Part I

Part I — Numbers Have Secrets

1 Chapter 1: Prime Numbers — The Atoms of Arithmetic

Placeholder — content coming soon.

2 Chapter 2 — The Collatz Conjecture: Simple Rules, Deep Mystery

2.1 The Rule

Pick any positive integer n. Apply the following rule repeatedly:

$$C(n) = \begin{cases} n/2 & \text{if } n \text{ is even} \\ 3n+1 & \text{if } n \text{ is odd} \end{cases}$$

For example, starting from $n = 6$:

$$6 \to 3 \to 10 \to 5 \to 16 \to 8 \to 4 \to 2 \to 1$$

Starting from $n = 27$, the sequence takes 111 steps before reaching 1 — and peaks at 9232 along the way.

The conjecture: No matter what positive integer you start with, the sequence always eventually reaches 1.

This has been verified computationally for all starting values up to approximately $2^{68} \approx 2.95 \times 10^{20}$. No counterexample has ever been found. No proof has ever been found either. The mathematician Paul Erdős said of this problem: *"Mathematics is not yet ready for such problems."*

2.2 Implementing the Map

We begin by implementing the Collatz map and computing the full trajectory (the sequence of values) for a given starting point.

Code: Collatz trajectory

Mathematica

```
collatz[n_] := If[EvenQ[n], n/2, 3n + 1]

trajectory[n_] := NestWhileList[collatz, n, # != 1 &]

trajectory[27]
```

Python

```
def collatz(n):
    return n // 2 if n % 2 == 0 else 3 * n + 1

def trajectory(n):
    seq = [n]
    while seq[-1] != 1:
        seq.append(collatz(seq[-1]))
    return seq

trajectory(27)
```

Output:

```
[27, 82, 41, 124, 62, 31, 94, 47, 142, 71, 214, 107, 322, 161, 484, 242,
 121, 364, 182, 91, 274, 137, 412, 206, 103, 310, 155, 466, 233, 700, 350,
 175, 526, 263, 790, 395, 1186, 593, 1780, 890, 445, 1336, 668, 334, 167,
 502, 251, 754, 377, 1132, 566, 283, 850, 425, 1276, 638, 319, 958, 479,
 1438, 719, 2158, 1079, 3238, 1619, 4858, 2429, 7288, 3644, 1822, 911,
 2734, 1367, 4102, 2051, 6154, 3077, 9232, 4616, 2308, 1154, 577, 1732,
 866, 433, 1300, 650, 325, 976, 488, 244, 122, 61, 184, 92, 46, 23, 70,
 35, 106, 53, 160, 80, 40, 20, 10, 5, 16, 8, 4, 2, 1]
```

Notice that the sequence reaches a maximum of 9232 before descending to 1 — over 340 times the starting value.

2.3 Visualizing Trajectories

A plot of the trajectory reveals the characteristic shape: a jagged ascent, a dramatic peak, and a staircase descent.

Code: Plot trajectory for n = 27

Mathematica

```
traj = trajectory[27];

ListLinePlot[traj,
  PlotLabel -> "Collatz trajectory: n = 27",
  AxesLabel -> {"Step", "Value"},
  PlotStyle -> Blue]
```

Python

```
import matplotlib.pyplot as plt

traj = trajectory(27)

plt.figure(figsize=(10, 4))
plt.plot(traj, color='steelblue')
plt.title("Collatz trajectory: n = 27")
plt.xlabel("Step")
plt.ylabel("Value")
plt.tight_layout()
plt.show()
```

Output:

[Plot: x-axis = step number (0–111), y-axis = value. Jagged climb to peak ~9232 near step 77, then rapid descent to 1.]

We can also overlay trajectories for multiple starting values to compare their shapes.

Code: Overlay trajectories for n = 1 to 20

Mathematica

```
trajs = trajectory /@ Range[1, 20];

ListLinePlot[trajs,
  PlotLabel -> "Collatz trajectories: n = 1 to 20",
  AxesLabel -> {"Step", "Value"},
  PlotLegends -> Range[1, 20]]
```

Python

```
trajs = [trajectory(n) for n in range(1, 21)]

plt.figure(figsize=(10, 5))
for n, traj in enumerate(trajs, start=1):
    plt.plot(traj, alpha=0.6, label=str(n))
plt.title("Collatz trajectories: n = 1 to 20")
plt.xlabel("Step")
plt.ylabel("Value")
plt.legend(fontsize=7, ncol=4)
plt.tight_layout()
plt.show()
```

Output:

[Plot: 20 overlapping trajectories, varying lengths and peak heights. Most terminate within 20 steps; n=18 and n=19 are notably longer.]

2.4 Stopping Time

The **stopping time** of n is the number of steps required to reach 1. It is the length of the trajectory minus 1.

Code: Stopping times for n = 1 to 1000

Mathematica

```
stoppingTime[n_] := Length[trajectory[n]] - 1

times = Table[stoppingTime[n], {n, 1, 1000}];

ListPlot[times,
  PlotLabel -> "Stopping times: n = 1 to 1000",
  AxesLabel -> {"n", "Stopping time"},
  PlotStyle -> {PointSize[0.004], Gray}]
```

Python

```
def stopping_time(n):
    return len(trajectory(n)) - 1

times = [stopping_time(n) for n in range(1, 1001)]

plt.figure(figsize=(10, 4))
plt.scatter(range(1, 1001), times, s=1, color='gray', alpha=0.7)
plt.title("Stopping times: n = 1 to 1000")
plt.xlabel("n")
plt.ylabel("Stopping time")
plt.tight_layout()
plt.show()
```

Output:

[Scatter plot: irregular pattern, with visible vertical striping. Values range from 0 (n=1) to about 178 (n=871). No obvious trend, but values near powers of 2 tend to be low.]

The scatter plot looks almost random, yet it has structure. Notice that stopping times near powers of 2 tend to be especially low — why? Because powers of 2 divide repeatedly by 2 and reach 1 quickly, and numbers close to a power of 2 often inherit this shortcut.

Let us find the record-breakers: values of n whose stopping time exceeds that of every smaller n.

Code: Stopping time records up to n = 1000

Mathematica

```
records = {};
maxSoFar = 0;
Do[
  t = stoppingTime[n];
  If[t > maxSoFar,
    AppendTo[records, {n, t}];
    maxSoFar = t],
  {n, 1, 1000}];

records
```

Python

```
records = []
max_so_far = 0

for n in range(1, 1001):
    t = stopping_time(n)
    if t > max_so_far:
        records.append((n, t))
        max_so_far = t

records
```

Output:

```
{(1, 0), (2, 1), (3, 7), (6, 8), (7, 16), (9, 19), (18, 20), (25, 23),
 (27, 111), (54, 112), (73, 115), (97, 118), (129, 121), (171, 124),
 (231, 127), (313, 130), (327, 143), (649, 144), (703, 170), (871, 178)}
```

Observation: The record-holders are not random. Several are near-multiples of earlier record-holders, and many are odd. This is Research Topic 3.

2.5 The Collatz Tree

Instead of iterating the map *forward* ($n \to C(n)$), we can ask: which values lead *to* n?

The **inverse map** has two branches: - $2n$ always maps to n (the even branch). - $(n-1)/3$ maps to n if $(n-1)$ is divisible by 3 and $(n-1)/3$ is odd (the odd branch).

Starting from 1 and applying the inverse map recursively builds the **Collatz tree** — a tree whose root is 1 and whose branches contain every positive integer (if the conjecture is true).

Code: Build and display the Collatz tree (depth 6)

Mathematica

```
collatzChildren[n_] := Module[{children},
  children = {2n};
  If[Mod[n - 1, 3] == 0 && OddQ[(n - 1)/3] && (n - 1)/3 > 0,
    AppendTo[children, (n - 1)/3]];
  children]

tree = TreeGraph[
  Flatten[Table[
    Rule[n, #] & /@ collatzChildren[n],
    {n, Flatten[NestList[
      Function[level, Flatten[collatzChildren /@ level]],
      {1}, 5]]}]],
  VertexLabels -> "Name",
  GraphLayout -> "LayeredDigraphEmbedding"]

tree
```

Python

```
import networkx as nx
import matplotlib.pyplot as plt

def collatz_children(n):
    children = [2 * n]
    candidate = (n - 1) // 3
    if (n - 1) % 3 == 0 and candidate % 2 == 1 and candidate > 0:
        children.append(candidate)
    return children

def build_tree(root, depth):
    G = nx.DiGraph()
    frontier = [root]
    for _ in range(depth):
        next_frontier = []
        for node in frontier:
            for child in collatz_children(node):
                G.add_edge(node, child)
                next_frontier.append(child)
        frontier = next_frontier
    return G

G = build_tree(1, depth=6)

pos = nx.nx_agraph.graphviz_layout(G, prog="dot")
plt.figure(figsize=(14, 7))
nx.draw(G, pos, with_labels=True, node_size=400,
        font_size=7, arrows=True)
plt.title("Collatz Tree (depth 6)")
plt.tight_layout()
plt.show()
```

Output:

[Tree diagram rooted at 1, branching outward to depth 6. Most nodes have only one child (the doubling branch); occasional nodes have two children where the odd-branch inverse applies. The tree has a fractal-like asymmetric branching structure.]

The tree grows approximately by a factor of 2 at each level (since the even branch always exists, the odd branch only sometimes). This is related to why the conjecture is hard to disprove: the tree appears to eventually catch every positive integer, but proving it does is another matter.

2.6 Further Research Topics

2.6.1 Topic 1 — The $5n + 1$ Variant

Replace the rule $3n + 1$ with $5n + 1$: if n is odd, compute $5n + 1$; if n is even, divide by 2.

For example, starting from $n = 3$:

$$3 \to 16 \to 8 \to 4 \to 2 \to 1$$

But not all starting values reach 1. Some enter cycles. For example:

$$13 \to 66 \to 33 \to 166 \to 83 \to 416 \to 208 \to 104 \to 52 \to 26 \to 13 \to \cdots$$

This is a cycle of length 10.

Your tasks:

1. For all starting values $n \leq 10000$, determine whether the $5n + 1$ sequence terminates at 1, enters a known cycle, or appears to diverge.
2. Find all distinct cycles up to length 50. For each cycle, report its minimum element and its length.
3. Map the "basin of attraction" of each cycle: which starting values flow into which cycle?
4. Conjecture: what arithmetic property of n (e.g., residue mod some integer) predicts which cycle it enters?

Code: Detect termination or cycles in 5n+1

Mathematica

```
collatz5[n_] := If[EvenQ[n], n/2, 5n + 1]

classify[n_, maxSteps_: 10000] := Module[{seen, current},
  seen = <||>;
  current = n;
  Do[
    If[current == 1, Return["terminates"]];
    If[KeyExistsQ[seen, current], Return[{"cycle", current}]];
    seen[current] = i;
    current = collatz5[current],
    {i, maxSteps}];
  "diverges?"]

Table[{n, classify[n]}, {n, 1, 30}]
```

Python

```
def collatz5(n):
    return n // 2 if n % 2 == 0 else 5 * n + 1

def classify(n, max_steps=10000):
    seen = {}
    current = n
    for i in range(max_steps):
        if current == 1:
            return "terminates"
        if current in seen:
            return ("cycle", current)
        seen[current] = i
        current = collatz5(current)
    return "diverges?"

[(n, classify(n)) for n in range(1, 31)]
```

Output:

[Table of (n, classification) for n = 1..30. Most entries show "cycle" with a cycle-entry point; a few show "terminates".]

2.6.2 Topic 2 — Variants $3n+3$ and $3n+5$

The $3n+1$ rule can be generalized. Consider:

- **Variant A:** $3n+3$ when odd, $n/2$ when even.
- **Variant B:** $3n+5$ when odd, $n/2$ when even.

Note that $3n+3 = 3(n+1)$. Since n is odd, $n+1$ is even, so the next step immediately divides by 2. This makes Variant A structurally simpler — can you predict its behavior analytically?

Your tasks:

1. For each variant, compute trajectories for $n = 1$ to 200. Do they always terminate?
2. Find all cycles for each variant.
3. Compare stopping-time plots for the original, Variant A, and Variant B side by side.
4. For Variant A, prove (or give strong computational evidence for) the claim: *every trajectory eventually enters the cycle* $\{3, 6, 3, 6, ...\}$.

Code: Trajectories for variants A and B

Mathematica

```
variantA[n_] := If[EvenQ[n], n/2, 3n + 3]
variantB[n_] := If[EvenQ[n], n/2, 3n + 5]

trajA[n_] := NestWhileList[variantA, n,
  (#1 != #2 &), 2, 500]
trajB[n_] := NestWhileList[variantB, n,
  (#1 != #2 &), 2, 500]

Table[{n, Length[trajA[n]], Length[trajB[n]]},
  {n, 1, 20}]
```

Python

```
def variant_a(n):
    return n // 2 if n % 2 == 0 else 3 * n + 3

def variant_b(n):
    return n // 2 if n % 2 == 0 else 3 * n + 5

def traj_until_cycle(f, n, max_steps=500):
    seen = set()
    seq = [n]
    for _ in range(max_steps):
        if seq[-1] in seen:
            break
        seen.add(seq[-1])
        seq.append(f(seq[-1]))
    return seq

[(n, len(traj_until_cycle(variant_a, n)),
     len(traj_until_cycle(variant_b, n)))
 for n in range(1, 21)]
```

Output:

[Table: columns n, length of Variant A trajectory, length of Variant B trajectory. Variant A trajectories are notably short.]

2.6.3 Topic 3 — Delay Records

A **delay record** is a value of n whose stopping time is strictly greater than the stopping time of every positive integer less than n.

From the output above, the first several delay records are: $1, 2, 3, 6, 7, 9, 18, 25, 27, \ldots$

Your tasks:

1. Compute the first 50 delay records. For each, record n, its stopping time, and $\lfloor \log_2 n \rfloor$.
2. Plot stopping time vs. $\lfloor \log_2 n \rfloor$ for the delay records. Does stopping time grow roughly linearly with $\log_2 n$?
3. Among the delay records, what fraction are odd? What fraction are of the form $3k$ or $3k+1$ or $3k+2$?
4. Several delay records appear to be related by the formula $n' \approx 2n$ or $n' \approx 3n/2$. Investigate whether consecutive delay records obey a predictable multiplicative relationship.

Code: First 50 delay records

Mathematica

```
delayRecords = {};
maxSoFar = -1;
n = 1;
While[Length[delayRecords] < 50,
  t = stoppingTime[n];
  If[t > maxSoFar,
    AppendTo[delayRecords,
      {n, t, Floor[Log[2, n]]}];
    maxSoFar = t];
  n++];

delayRecords // TableForm
```

Python

```
import math

delay_records = []
max_so_far = -1
n = 1

while len(delay_records) < 50:
    t = stopping_time(n)
    if t > max_so_far:
        delay_records.append(
            (n, t, int(math.log2(n)) if n > 0 else 0))
        max_so_far = t
    n += 1

delay_records
```

Output:

[Table of 50 rows: (n, stopping time, floor(log2 n)). Stopping times grow roughly in proportion to log2(n), with notable jumps at n=27, n=703, n=871.]

2.6.4 Topic 4 — Structure in the Stopping-Time Plot

The scatter plot of stopping times vs. n (from the main chapter) has a layered, self-similar appearance. Look carefully at the plot for n up to 10^6: the points do not fill the plane uniformly. They cluster along several "rays" or "bands."

Your tasks:

1. Generate the stopping-time scatter plot for $n = 1$ to 10^6. Use a small point size.
2. Identify at least 3 distinct bands visually. For each band, sample 20 values of n that lie on it and examine their binary representations. What do they have in common?
3. Conjecture: the band structure is related to the highest power of 2 dividing n, or to n mod 2^k for some k. Test your conjecture.
4. Color-code points by $n \mod 3$ (i.e., residue 0, 1, or 2 mod 3). Does the color pattern align with the band structure?

Code: Stopping-time plot for n up to 10^6

Mathematica

```
(* This may take a few minutes *)
times = Table[stoppingTime[n], {n, 1, 10^6}];

ListPlot[times,
  PlotStyle -> {PointSize[0.0003],
    ColorData["Rainbow"][#/(Max[times])] & },
  PlotLabel -> "Stopping times: n = 1 to 10^6",
  AxesLabel -> {"n", "Stopping time"}]
```

Python

```python
import numpy as np

N = 10**6
times = np.array([stopping_time(n) for n in range(1, N + 1)])

plt.figure(figsize=(14, 5))
plt.scatter(range(1, N + 1), times,
            s=0.01, c=times, cmap='rainbow',
            alpha=0.5)
plt.colorbar(label='Stopping time')
plt.title("Stopping times: n = 1 to 10^6")
plt.xlabel("n")
plt.ylabel("Stopping time")
plt.tight_layout()
plt.show()
```

Output:

[Dense scatter plot showing clear diagonal band structure. Longest stopping times cluster around $n \approx 837799$ (stopping time 524).]

> **Tip:** This computation takes time for n up to 10^6 in pure Python. Consider using NumPy vectorization or caching intermediate results to speed it up.

2.6.5 Topic 5 — The Generalized (p, q)-Collatz Map

The original Collatz map uses $p = 2$ (divide even numbers by 2) and $q = 3$ (multiply odd numbers by 3 and add 1). We can generalize:

$$C_{p,q}(n) = \begin{cases} n/p & \text{if } p \mid n \\ qn + 1 & \text{otherwise} \end{cases}$$

Your tasks:

1. For each pair (p, q) with $p, q \in \{2, 3, 4, 5\}$ and $p \neq q$, test whether the map appears to always terminate (reach a fixed point or a cycle containing 1) for all starting values $n \leq 1000$.
2. Build a 4×4 table (rows = p, columns = q) where each cell contains: "terminates", "cycles", or "diverges?" based on your experiments.

3. For pairs where the map terminates, report the average stopping time. Does average stopping time increase or decrease with q/p?
4. Conjecture a necessary condition on the ratio q/p for the map to always terminate. (Hint: consider what happens when $q/p > 1$ vs. $q/p < 1$.)

Code: Classify (p,q)-Collatz maps

Mathematica

```
collatzPQ[p_, q_][n_] :=
  If[Mod[n, p] == 0, n/p, q*n + 1]

classifyPQ[p_, q_, maxN_: 200, maxSteps_: 1000] :=
  Module[{results},
    results = Table[
      Module[{seen, current},
        seen = <||>;
        current = n;
        Catch[
          Do[
            If[current == 1, Throw["terminates"]];
            If[KeyExistsQ[seen, current],
              Throw[{"cycle", current}]];
            seen[current] = i;
            current = collatzPQ[p, q][current],
            {i, maxSteps}];
          "diverges?"]],
      {n, 1, maxN}];
    results]

Table[
  {p, q, Tally[classifyPQ[p, q]]},
  {p, {2, 3, 4, 5}},
  {q, {2, 3, 4, 5}}]
```

Python

```
def collatz_pq(p, q, n):
    return n // p if n % p == 0 else q * n + 1

def classify_pq(p, q, max_n=200, max_steps=1000):
    results = []
    for n in range(1, max_n + 1):
        seen = {}
        current = n
        outcome = "diverges?"
        for i in range(max_steps):
            if current == 1:
                outcome = "terminates"
                break
            if current in seen:
                outcome = ("cycle", current)
                break
            seen[current] = i
            current = collatz_pq(p, q, current)
        results.append(outcome)
    return results

from collections import Counter

table = {}
for p in [2, 3, 4, 5]:
    for q in [2, 3, 4, 5]:
        if p != q:
            res = classify_pq(p, q)
            table[(p, q)] = Counter(
                r if isinstance(r, str) else r[0]
                for r in res)

table
```

Output:

[Dictionary/table of outcome tallies for each (p,q) pair. Pairs with $q/p < 1$ (e.g., $p=5$, $q=2$) tend to terminate; pairs with large q/p (e.g., $p=2$, $q=5$) produce many cycles or apparent divergence.]

3 Chapter 3: Fibonacci Numbers and Their Cousins

Placeholder — content coming soon.

4 Chapter 4: Digits, Bases, and Normal Numbers

Placeholder — content coming soon.

5 Chapter 5: Pascal's Triangle — Infinite Patterns in a Simple Array

Placeholder — content coming soon.

6 Chapter 6: Continued Fractions — The Most Natural Way to Write Numbers

Placeholder — content coming soon.

7 Chapter 7: Integer Sequences and the OEIS

Placeholder — content coming soon.

Part II

Part II — Structure and Dynamics

8 Chapter 8: Cellular Automata — Rules That Generate Worlds

Placeholder — content coming soon.

9 Chapter 9: Chaos in a Simple Formula — The Logistic Map

Placeholder — content coming soon.

Part III

Part III — Geometry Through Computation

10 Chapter 10: Fractal Geometry — Infinite Complexity from Simple Rules

Placeholder — content coming soon.

11 Chapter 11: Tilings and Symmetry

Placeholder — content coming soon.

Part IV

Part IV — The Experimental Method

12 Chapter 12: When the Computer Misleads You

Placeholder — content coming soon.

13 Chapter 13: From Conjecture to Communication

Placeholder — content coming soon.

Part V

Appendices

14 Appendix A: Mathematica Quick Reference

Placeholder — content coming soon.

15 Appendix B: Python Quick Reference

Placeholder — content coming soon.

16 Appendix C: Guide to the OEIS

Placeholder — content coming soon.

17 Appendix D: Further Reading

Placeholder — content coming soon.